

INDEX

S/N	AIM
1	To study and implement various DDL commands to create and modify a table in MySQL.
2	To study and implement various DML commands to insert, retrieve, update, and delete data.
3	To study and implement DCL commands to control access and permissions in MySQL.
4	To implement aggregate functions MIN(), MAX(), SUM(), AVG(), and COUNT() on a table in MySQL.
5	To perform data retrieval using conditions (greater than) and sorting techniques using ASC and DESC in MySQL.
6	To implement NATURAL JOIN and EQUI JOIN on tables in MySQL.
7	To implement LEFT OUTER JOIN and RIGHT OUTER JOIN on tables in MySQL.
8	To implement SELF JOIN on a table in MySQL.

Q1 . To study and implement various DDL commands to create and modify a table in MySQL

Experiment 1

Aim

To study and implement various **DDL (Data Definition Language)** commands to create and modify a table in MySQL.

Theory

DDL (Data Definition Language) commands are used to define, manage, and modify the structure of database objects such as tables, databases, indexes, etc. These commands are essential for designing the database schema.

DDL commands do not deal with the data inside the table, but instead focus on the structure of the database.

Common DDL Commands

- **CREATE** → Used to create a new database or table
- **ALTER** → Used to modify the structure of an existing table
- **DROP** → Used to delete a table or database permanently
- **TRUNCATE** → Used to remove all records from a table quickly

Explanation of Queries

- Database **college** is created and selected.
- A table **student** is created with fields: id, name, age, and course.
- DESC command displays the table structure.
- ALTER TABLE ADD adds new column **marks**.
- ALTER TABLE MODIFY increases the size of the **name** column.

Result

The DDL commands were successfully executed. The table was created and modified using different DDL operations in MySQL.

SQL Queries (Implementation)

-- Create Database

```
CREATE DATABASE EXPERIMENT_1;
```

-- Select Database

```
USE EXPERIMENT_1;
```

-- Create Table

```
CREATE TABLE student (
```

```
    id INT PRIMARY KEY,
```

```
    name VARCHAR(50),
```

```
    age INT,
```

```
    course VARCHAR(50)
```

```
);
```

-- Display Table Structure

```
DESC student;
```

-- Add New Column

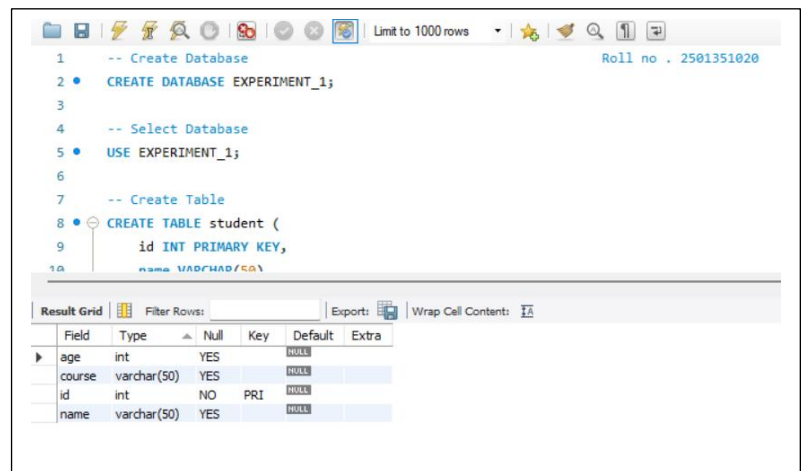
```
ALTER TABLE student
```

```
ADD marks INT;
```

-- Modify Existing Column

```
ALTER TABLE student
```

```
MODIFY name VARCHAR(100);
```



```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| college_database |
| company_database |
| experiment_1 |
| information_schema |
| mysql |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
9 rows in set (0.00 sec)

mysql> USE experiment_1;
Database changed
mysql> show tables;
+-----+
| Tables_in_experiment_1 |
+-----+
| student |
+-----+
1 row in set (0.00 sec)
```

Conclusion

DDL commands play a vital role in database design. They help in creating and modifying the structure of database objects efficiently.

Q2. To study and implement various DML commands to insert, retrieve, update, and delete data.

Experiment 2

Aim

To study and implement various **DML (Data Manipulation Language)** commands to insert, retrieve, update, and delete data in MySQL.

Theory

DML (Data Manipulation Language) commands are used to manage and manipulate data stored in database tables. These commands help in inserting new records, retrieving existing data, updating records, and deleting unwanted data.

Common DML Commands

- **INSERT** → To add new data into a table
- **SELECT** → To retrieve data from a table
- **UPDATE** → To modify existing data
- **DELETE** → To remove data from a table

SQL Queries (Implementation)

```
-- Use Database                                || EXPERIMENT_2 ||
```

```
USE EXPERIMENT_1;
```

```
-- Insert Data
```

```
INSERT INTO student (id, name, age, course, marks) VALUES
```

```
(1, 'Rajveer', 20, 'BTech', 85),
```

```
(2, 'shruti', 21, 'BA', 90),
```

```
(3, 'veerraj', 19, 'B.Tech', 78), <----- DELETE THAT DATA
```

```
(4, 'Badal', 19, 'B.Tech', 78);
```

-- Retrieve Data

SELECT * FROM student;

-- Update Data

UPDATE student

SET marks = 95

WHERE id = 2;

-- Delete Data

DELETE FROM student

WHERE id = 3;

```
mysql> USE database_name;
ERROR 1049 (42000): Unknown database 'database_name'
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| college_database |
| company_database |
| experiment_1 |
| information_schema |
| mysql |
| performance_schema |
| sakila |
| sys |
| world |
+-----+
9 rows in set (0.03 sec)

mysql> USE experiment_1;
Database changed
mysql> show tables;
+-----+
| Tables_in_experiment_1 |
+-----+
| student |
+-----+
1 row in set (0.01 sec)

mysql> SELECT * FROM student;
+----+-----+-----+-----+-----+
| id | name   | age | course | marks |
+----+-----+-----+-----+-----+
| 1  | Rajveer | 20  | BTech  | 85    |
| 2  | shruti  | 21  | BA     | 95    |
| 4  | Badal  | 19  | B.Tech | 78    |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Explanation of Queries

- INSERT adds new records into the **student** table
- SELECT displays all records from the table
- UPDATE modifies marks of a specific student
- DELETE removes a record from the table

Result

The DML commands were successfully executed. Data was inserted, retrieved, updated, and deleted from the table.

```
mysql> SELECT * FROM student;
+----+-----+-----+-----+-----+
| id | name   | age | course | marks |
+----+-----+-----+-----+-----+
| 1  | Rajveer | 20  | BTech  | 85    |
| 2  | shruti  | 21  | BA     | 95    |
| 4  | Badal  | 19  | B.Tech | 78    |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Result Grid					
Filter Rows:					
Edit:    Export/Imp					
	id	name	age	course	marks
▶	1	Rajveer	20	BTech	85
	2	shruti	21	BA	95
	4	Badal	19	B.Tech	78
✱	NULL	NULL	NULL	NULL	NULL

Conclusion

DML commands are used to handle data inside tables. They allow users to efficiently manage and manipulate records in a database

Q3. To study and implement DCL commands to control access and permissions in MySQL.

Experiment 3

Aim

To study and implement **DCL (Data Control Language)** commands to control access and permissions in MySQL.

Theory

DCL (Data Control Language) commands are used to control access to data stored in a database. These commands help in providing security by granting or removing user permissions on database objects like tables and databases.

DCL ensures that only authorized users can perform specific operations such as SELECT, INSERT, UPDATE, or DELETE.

Common DCL Commands

- **GRANT** → To give permissions to a user
- **REVOKE** → To remove permissions from a user

SQL Queries (Implementation)

```
-- || EXPERIMENT 3 || |
```

```
-- Step 1: Remove user if it already exists (to avoid error)
```

```
DROP USER IF EXISTS 'student_user'@'localhost';
```

```
-- Step 2: Create a new user with username and password
```

```
CREATE USER 'student_user'@'localhost' IDENTIFIED BY 'Rajveer123';
```

```
-- Step 3: Grant permissions (SELECT and INSERT) on EXPERIMENT_1 database
```

```
GRANT SELECT, INSERT ON EXPERIMENT_1.* TO 'student_user'@'localhost';
```

```
-- Step 4: Apply and refresh privileges
```

```
FLUSH PRIVILEGES;
```

```
-- Step 5: Revoke INSERT permission from the user
```

```
REVOKE INSERT ON EXPERIMENT_1.* FROM 'student_user'@'localhost';
```


-- Step 6: Check final permissions of the user (for verification)

```
SHOW GRANTS FOR 'student_user'@'localhost';
```

Explanation of Queries

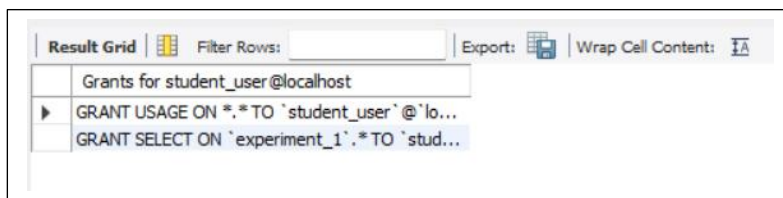
- CREATE USER creates a new database user
- GRANT gives specific permissions (SELECT, INSERT)
- FLUSH PRIVILEGES updates the permission table
- REVOKE removes previously granted permissions

Result

The DCL commands were successfully executed. User access was controlled by granting and revoking permissions.

Conclusion

DCL commands are important for database security. They help in controlling user access and maintaining data safety.



Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	Grants for student_user@localhost			
▶	GRANT USAGE ON *.* TO 'student_user'@'lo...			
	GRANT SELECT ON 'experiment_1'.* TO 'stud...			

```
mysql> SHOW GRANTS FOR 'student_user'@'localhost';
+-----+
| Grants for student_user@localhost |
+-----+
| GRANT USAGE ON *.* TO 'student_user'@'localhost' |
| GRANT SELECT ON 'experiment_1'.* TO 'student_user'@'localhost' |
+-----+
2 rows in set (0.01 sec)
```

Q4. To implement aggregate functions MIN(), MAX(), SUM(), AVG(), and COUNT() on a table in MySQL.

Experiment 4

Aim

To implement aggregate functions **MIN()**, **MAX()**, **SUM()**, **AVG()**, and **COUNT()** on a table in MySQL.

Theory

Aggregate functions are used to perform calculations on a set of values in a table and return a single result. These functions are very useful for data analysis and summarization in databases.

They work on numeric data and help in finding minimum, maximum, total, average, and number of records.

Aggregate Functions

- **MIN()** → Returns the smallest value
- **MAX()** → Returns the largest value
- **SUM()** → Returns total sum of values
- **AVG()** → Returns average value
- **COUNT()** → Returns total number of records

SQL Queries (Implementation)

```
-- || EXPERIMENT 4 : AGGREGATE FUNCTIONS ||
```

```
-- Step 1: Use Database  
USE EXPERIMENT_1;
```

```
-- Step 2: Display Table Data  
SELECT * FROM student;
```

```
-- Step 3: Find Minimum Marks  
SELECT MIN(marks) AS Minimum_Marks FROM student;
```

-- Step 4: Find Maximum Marks

```
SELECT MAX(marks) AS Maximum_Marks FROM student;
```

-- Step 5: Find Total Marks

```
SELECT SUM(marks) AS Total_Marks FROM student;
```

-- Step 6: Find Average Marks

```
SELECT AVG(marks) AS Average_Marks FROM student;
```

-- Step 7: Count Total Students

```
SELECT COUNT(*) AS Total_Students FROM student;
```

SELECT

MIN(marks) AS Minimum_Marks,

MAX(marks) AS Maximum_Marks,

SUM(marks) AS Total_Marks,

AVG(marks) AS Average_Marks,

COUNT(*) AS Total_Students

```
mysql> SELECT
-> MIN(marks) AS Minimum_Marks,
-> MAX(marks) AS Maximum_Marks,
-> SUM(marks) AS Total_Marks,
-> AVG(marks) AS Average_Marks,
-> COUNT(*) AS Total_Students
-> FROM student;
+-----+-----+-----+-----+-----+
| Minimum_Marks | Maximum_Marks | Total_Marks | Average_Marks | Total_Students |
+-----+-----+-----+-----+-----+
| 78            | 95            | 258         | 86.0000       | 3              |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

FROM student;

Minimum_Marks	Maximum_Marks	Total_Marks	Average_Marks	Total_Students
78	95	258	86.0000	3

Explanation

- MIN(marks) → gives lowest marks
- MAX(marks) → gives highest marks
- SUM(marks) → adds all marks
- AVG(marks) → calculates average
- COUNT(*) → counts total rows

Result

All aggregate functions were successfully executed on the **student** table, and the required results were obtained.

Conclusion

Aggregate functions simplify data analysis by providing summarized results from large datasets in a database.

Q5. To perform data retrieval using conditions (greater than) and sorting techniques using ASC and DESC in MySQL.

Experiment 5

◆ Aim

To perform data retrieval using **conditions (greater than)** and sorting techniques using **ASC and DESC** in MySQL.

Theory

Data retrieval in MySQL is done using the SELECT statement.

Conditions can be applied using the WHERE clause to filter specific records. Sorting of data is done using the ORDER BY clause.

- WHERE → filters records based on conditions
- ORDER BY → sorts data
 - **ASC** → Ascending order (default)
 - **DESC** → Descending order

SQL Queries (Implementation)

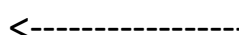
-- || EXPERIMENT 5 : CONDITIONS & SORTING ||

-- Use Database

USE EXPERIMENT_1;

-- Display All Data

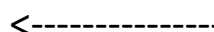
SELECT * FROM student;



	id	name	age	course	marks
▶	1	Rajveer	20	BTech	85
	2	shruti	21	BA	95
	4	Badal	19	B.Tech	78
•	NULL	NULL	NULL	NULL	NULL

-- Condition: Marks Greater Than 80

SELECT * FROM student



	id	name	age	course	marks
▶	1	Rajveer	20	BTech	85
	2	shruti	21	BA	95
•	NULL	NULL	NULL	NULL	NULL

WHERE marks > 80;

-- Sorting in Ascending Order

SELECT * FROM student

ORDER BY marks ASC;

<-----

id	name	age	course	marks
1	Rajveer	20	BTech	85
2	shruti	21	BA	95
NULL	NULL	NULL	NULL	NULL

-- Sorting in Descending Order

SELECT * FROM student

ORDER BY marks DESC;

<-----

id	name	age	course	marks
4	Badal	19	B.Tech	78
1	Rajveer	20	BTech	85
2	shruti	21	BA	95
NULL	NULL	NULL	NULL	NULL

-- Condition with Descending Sorting

SELECT * FROM student

WHERE marks > 80

ORDER BY marks DESC;

<-----

id	name	age	course	marks
2	shruti	21	BA	95
1	Rajveer	20	BTech	85
4	Badal	19	B.Tech	78
NULL	NULL	NULL	NULL	NULL

Explanation

- WHERE marks > 80 → selects students with marks greater than 80
- ORDER BY marks ASC → sorts marks from low to high
- ORDER BY marks DESC → sorts marks from high to low
- Combined query filters + sorts results together

Result

The data was successfully retrieved using conditions and sorted using ascending and descending order.

Conclusion

The WHERE and ORDER BY clauses are essential for filtering and organizing data efficiently in MySQL.

Q6. To implement NATURAL JOIN and EQUI JOIN on tables in MySQL.

Experiment 6

Aim

To implement **NATURAL JOIN** and **EQUI JOIN** on tables in MySQL.

Theory

Joins in MySQL are used to combine data from two or more tables based on a related column.

- **NATURAL JOIN** → Automatically joins tables based on columns with the same name
- **EQUI JOIN** → Joins tables using equality condition (=) in ON clause

SQL Queries (Implementation)

```
-- || EXPERIMENT 6 : NATURAL JOIN & EQUI JOIN ||
```

```
USE EXPERIMENT_1;
```

```
-- Create Table 1: student_id
```

```
CREATE TABLE student_id (
```

```
    id INT PRIMARY KEY,
```

```
    name VARCHAR(50),
```

```
    course VARCHAR(50)
```

```
);
```

-- Create Table 2: marks

CREATE TABLE marks (

id INT,

marks INT

);

-- Insert Data into student_id

INSERT INTO student_id VALUES

(1, 'Rajveer', 'BTech'),

(2, 'Badal', 'BTech'),

(3, 'Shruti', 'BA'),

(4, 'Aditya', 'BBA'),

(5, 'Shiv', 'BTech');

-- Insert Data into marks

INSERT INTO marks VALUES

(1, 85),

(2, 90),

(3, 78),

(4, 98),

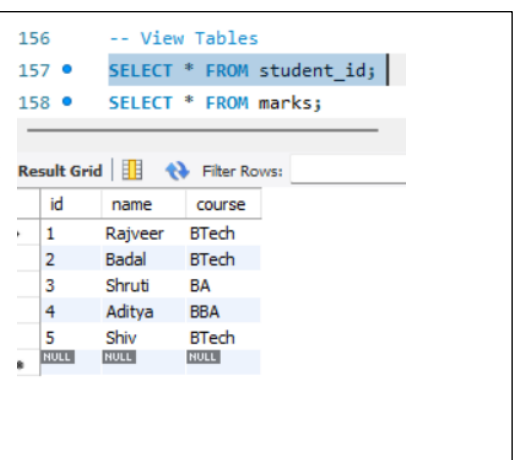
(5, 88);

-- View Tables

SELECT * FROM student_id;

SELECT * FROM marks;

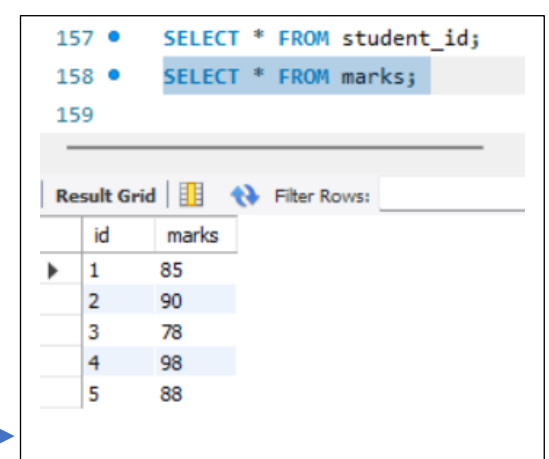
-- NATURAL JOIN



156 -- View Tables
157 • SELECT * FROM student_id;
158 • SELECT * FROM marks;

Result Grid | Filter Rows:

	id	name	course
1	1	Rajveer	BTech
2	2	Badal	BTech
3	3	Shruti	BA
4	4	Aditya	BBA
5	5	Shiv	BTech
	NULL	NULL	NULL



157 • SELECT * FROM student_id;
158 • SELECT * FROM marks;
159

Result Grid | Filter Rows:

	id	marks
▶	1	85
	2	90
	3	78
	4	98
	5	88

```
SELECT * FROM student_id
```

```
NATURAL JOIN marks;
```

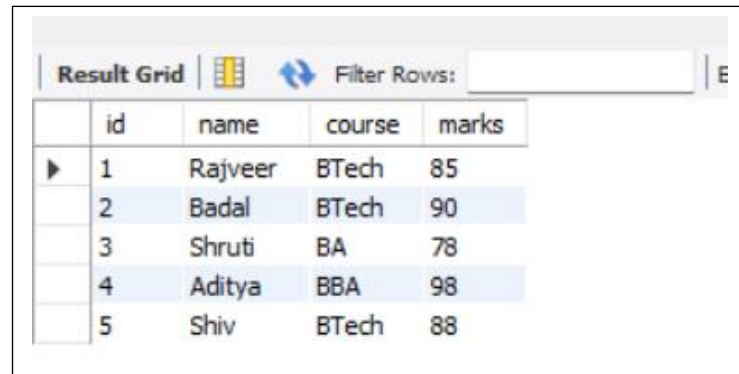
```
-- EQUI JOIN
```

```
SELECT student_id.id, student_id.name, student_id.course,  
marks.marks
```

```
FROM student_id
```

```
JOIN marks
```

```
ON student_id.id = marks.id;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' input field. Below the header, there is a table with 5 rows and 5 columns. The columns are labeled 'id', 'name', 'course', and 'marks'. The rows contain the following data: (1, Rajveer, BTech, 85), (2, Badal, BTech, 90), (3, Shruti, BA, 78), (4, Aditya, BBA, 98), and (5, Shiv, BTech, 88). The first row is highlighted with a blue background.

	id	name	course	marks
▶	1	Rajveer	BTech	85
	2	Badal	BTech	90
	3	Shruti	BA	78
	4	Aditya	BBA	98
	5	Shiv	BTech	88

Explanation

- NATURAL JOIN automatically joins using common column (id)
- EQUI JOIN uses ON student.id = marks.id condition
- Both produce similar results but work differently

Result

Data from both tables was successfully combined using NATURAL JOIN and EQUI JOIN.

Conclusion

Joins are important for combining related data from multiple tables. NATURAL JOIN is automatic, while EQUI JOIN provides more control.

Q7. To implement LEFT OUTER JOIN and RIGHT OUTER JOIN on tables in MySQL.

Experiment 7

Aim

To implement **LEFT OUTER JOIN** and **RIGHT OUTER JOIN** on tables in MySQL.

Theory

Outer joins are used to retrieve data from multiple tables including unmatched records.

- **LEFT OUTER JOIN** → Returns all records from the left table and matching records from the right table. If no match, NULL values are shown.
- **RIGHT OUTER JOIN** → Returns all records from the right table and matching records from the left table. If no match, NULL values are shown.

SQL Queries (Implementation)

```
-- || EXPERIMENT 7 : LEFT & RIGHT OUTER JOIN ||
```

```
-- Use Database
```

```
USE EXPERIMENT_1;
```

```
-- Drop tables if already exist (to avoid errors)
```

```
DROP TABLE IF EXISTS student_left;
```

```
DROP TABLE IF EXISTS marks_right;
```

-- Create Table 1: student_left

```
CREATE TABLE student_left (  
    id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

-- Create Table 2: marks_right

```
CREATE TABLE marks_right (  
    id INT,  
    marks INT  
);
```

-- Insert Data into student_left

```
INSERT INTO student_left VALUES  
(1, 'Rajveer'),  
(2, 'Badal'),  
(3, 'Shruti'),  
(4, 'Aditya'),  
(6, 'Shiv');
```

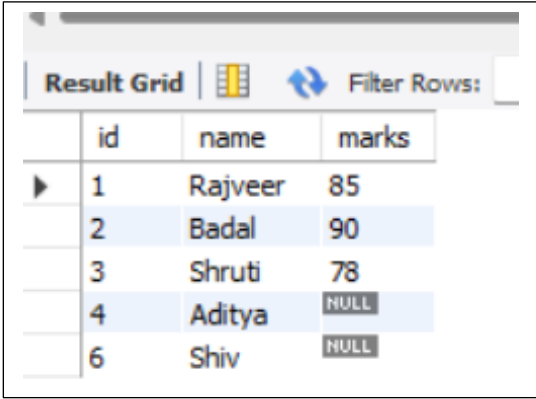
-- Insert Data into marks_right

```
INSERT INTO marks_right VALUES  
(1, 85),  
(2, 90),  
(3, 78),
```

(5, 88);

-- LEFT OUTER JOIN

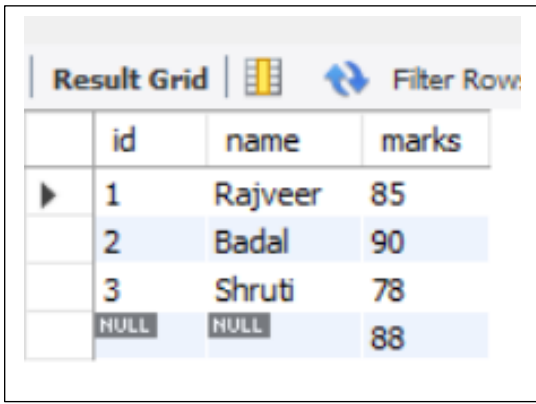
```
SELECT s.id, s.name, m.marks
FROM student_left s
LEFT JOIN marks_right m
ON s.id = m.id;
```



	id	name	marks
▶	1	Rajveer	85
	2	Badal	90
	3	Shruti	78
	4	Aditya	NULL
	6	Shiv	NULL

-- RIGHT OUTER JOIN

```
SELECT s.id, s.name, m.marks
FROM student_left s
RIGHT JOIN marks_right m
ON s.id = m.id;
```



	id	name	marks
▶	1	Rajveer	85
	2	Badal	90
	3	Shruti	78
	NULL	NULL	88

Explanation

- In **LEFT JOIN**, all students are shown (even if marks not present → NULL)
- In **RIGHT JOIN**, all marks records are shown (even if student not present → NULL)
- Unmatched rows clearly demonstrate outer join behavior

Result

The LEFT and RIGHT OUTER JOIN operations were successfully performed, displaying both matched and unmatched records.

Conclusion

Outer joins help retrieve complete data from tables, including unmatched records, which is useful in real-world database applications.

Q8. To implement SELF JOIN on a table in MySQL.

Experiment 8

Aim

To implement SELF JOIN on a table in MySQL.

Theory

A SELF JOIN is a type of join where a table is joined with itself. It is used when we want to compare rows within the same table.

To perform a SELF JOIN, we use table aliases (different names for the same table).

Example: Finding employee and their manager from the same table.

SQL Queries (Implementation)

```
-- || EXPERIMENT 8 : SELF JOIN ||
```

```
-- Use Database
```

```
USE EXPERIMENT_1;
```

```
-- Drop table if exists (to avoid error)
```

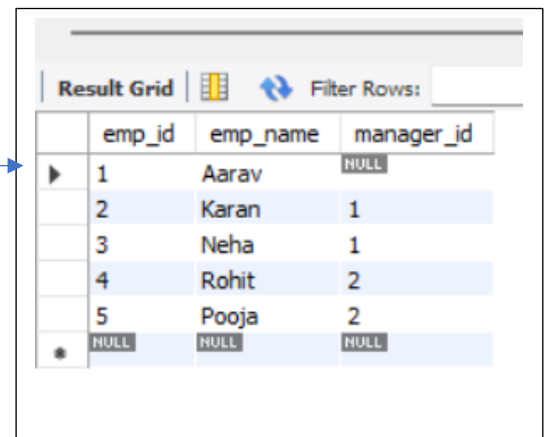
```
DROP TABLE IF EXISTS employee;
```

```
-- Create Table
```

```
CREATE TABLE employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    manager_id INT  
);
```

-- Insert Data

```
INSERT INTO employee VALUES
(1, 'Rajveer', NULL), -- Top manager
(2, 'Badal', 1),
(3, 'Shruti', 1),
(4, 'Aditya', 2),
(5, 'Shiv', 2);
select * from employee;
```

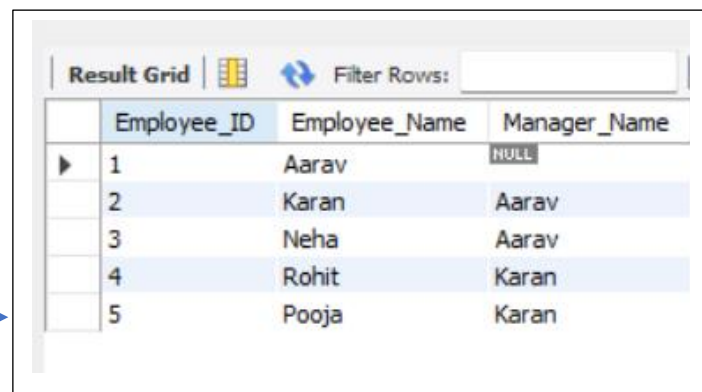


A screenshot of a database application's 'Result Grid'. It displays a table with three columns: 'emp_id', 'emp_name', and 'manager_id'. The data is as follows:

emp_id	emp_name	manager_id
1	Aarav	NULL
2	Karan	1
3	Neha	1
4	Rohit	2
5	Pooja	2
NULL	NULL	NULL

-- SELF JOIN (Employee with Manager)

```
SELECT
    e.emp_id AS Employee_ID,
    e.emp_name AS Employee_Name,
    m.emp_name AS Manager_Name
FROM employee e
LEFT JOIN employee m
ON e.manager_id = m.emp_id;
```



A screenshot of a database application's 'Result Grid' showing the output of a self-join query. The table has three columns: 'Employee_ID', 'Employee_Name', and 'Manager_Name'. The data is as follows:

Employee_ID	Employee_Name	Manager_Name
1	Aarav	NULL
2	Karan	Aarav
3	Neha	Aarav
4	Rohit	Karan
5	Pooja	Karan

Explanation

- Table employee is joined with itself
- **e** → represents employee
- **m** → represents manager
- manager_id links employee to manager

Result

The SELF JOIN successfully displayed employees along with their respective managers.

Conclusion

SELF JOIN is useful when relationships exist within the same table, such as employee-manager hierarchy.

```
mysql> SELECT * FROM marks;
```

id	marks
1	85
2	90
3	78
4	98
5	88

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM student_id;
```

id	name	course
1	Rajveer	BTech
2	Badal	BTech
3	Shruti	BA
4	Aditya	BBA
5	Shiv	BTech

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM student_left;
```

id	name
1	Rajveer
2	Badal
3	Shruti
4	Aditya
6	Shiv

```
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM marks_right;
```

id	marks
1	85
2	90
3	78
5	88

```
4 rows in set (0.00 sec)
```